

Aprendizado Profundo

Generative Adversarial Networks

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Visão Geral

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Generative Adversarial Network¹

Uma **GAN** é um modelo não supervisionado cujo objetivo é **gerar novas instâncias indistinguíveis** do conjunto de dados de treinamento.

- Não constrói explicitamente uma distribuição de probabilidade sobre os dados.
- Em vez disso, aprende a amostrar dessa distribuição.
- Usualmente é formada por **duas redes** treinadas em conjunto:
 - **Gerador**: mapeia uma distribuição aleatória para o espaço dos dados.
 - **Discriminador** (ou crítico): classifica as instâncias como reais ou sintéticas.
- Desenvolvida originalmente por Ian Goodfellow em 2014.

¹Ian Goodfellow et al. "Generative Adversarial Nets". Em: [Advances in Neural Information Processing Systems](#). Vol. 27. 2014.

Aplicações

Onde GANs já mostraram resultados marcantes:

- **Síntese de faces fotorrealistas:** *StyleGAN*² produz retratos indistinguíveis de pessoas reais.
- **Image-to-image translation:** *pix2pix*³ (mapa $\rightarrow$ foto, esboço $\rightarrow$ foto) e *CycleGAN*⁴ (cavalo $\leftrightarrow$ zebra) sem pares alinhados.
- **Super resolução:** ampliar imagens preservando textura plausível.
- **Aumento de dados:** gerar amostras sintéticas para classes raras em domínios médicos e industriais.
- **Deepfakes:** vídeo e áudio sintéticos de pessoas reais, com todas as questões éticas que isso traz.

²Tero Karras, Samuli Laine e Timo Aila. “A Style-Based Generator Architecture for Generative Adversarial Networks”. Em: [IEEE/CVF Conference on Computer Vision and Pattern Recognition \(CVPR\)](#). 2019.

³Phillip Isola et al. “Image-to-Image Translation with Conditional Adversarial Networks”. Em: [IEEE Conference on Computer Vision and Pattern Recognition \(CVPR\)](#). 2017.

⁴Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks”. Em: [IEEE International Conference on Computer Vision \(ICCV\)](#). 2017.

GANs – Princípios

Objetivo: gerar novas instâncias $\{x^{(j)*}\}$ que sigam a mesma distribuição dos dados de treino $\{x^{(i)}\}$.

Processo de geração de uma nova instância

1. Amostrar uma **variável latente** $z^{(j)}$ de uma distribuição simples (normal, uniforme, ...).
2. Usar $z^{(j)}$ como entrada para uma rede neural:

$$x^{(j)*} = g[z^{(j)}, \theta]$$

onde θ são os parâmetros do gerador aprendidos durante o treino.

GANs – Princípios

- Teremos atingido o objetivo quando as instâncias geradas forem **indistinguíveis** das instâncias do conjunto de dados.
- Introduzimos então uma segunda rede $f[\cdot, \phi]$ que tenta **classificar** suas entradas como reais ou geradas.
 - Se o discriminador consegue classificar bem, ele pode gerar um sinal de gradiente usado para melhorar o gerador.
 - Se o discriminador não consegue distinguir, o gerador atingiu o objetivo.

Intuição de jogo

O gerador tenta enganar o discriminador; o discriminador tenta não ser enganado. Um equilíbrio de Nash nesse jogo corresponde a $P_{\text{gerada}} = P_{\text{real}}$.

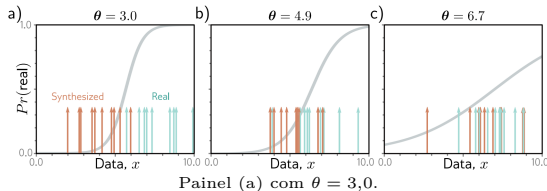
Para enxergar essa dinâmica, vamos olhar um exemplo unidimensional em que o gerador se reduz a um único parâmetro.

GANs – Princípios: um exemplo *toy*

- Os dados reais foram obtidos de uma normal $\mathcal{N}(7, 1)$.
- O latente é amostrado de $z^{(j)} \sim \mathcal{N}(0, 1)$ e o modelo gerador possui **um único parâmetro** que translada essa distribuição:

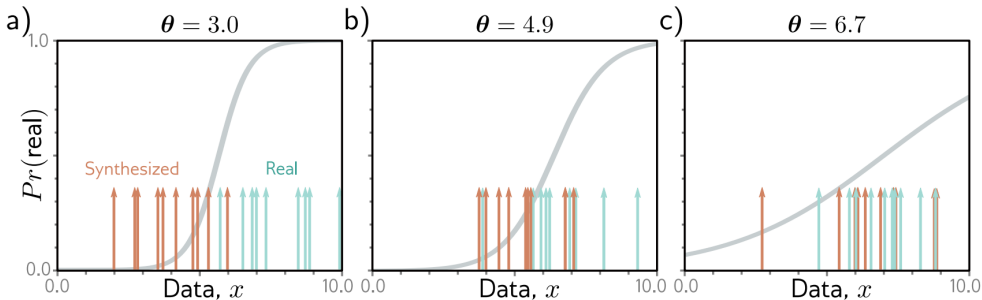
$$x^{(j)*} = z^{(j)} + \theta.$$

- A sigmóide representa o classificador binário ideal.



GANs – Princípios: sobreposição de distribuições

- Valores diferentes de θ fazem com que o classificador tenha maior ou menor dificuldade em separar as distribuições.
- Quando $\theta \approx 7$, as distribuições coincidem e o classificador ideal degenera para chance aleatória (sigmóide quase plana).



(a) distribuições separadas; (b) parcialmente sobrepostas; (c) quase coincidentes.

Overview

1. O que são GANs
- 2. MinMax Loss**
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Loss do Discriminador

Para o discriminador, temos um problema de **classificação binária**. Rotulamos as instâncias reais com $y = 1$ e as geradas com $y = 0$, e minimizamos a entropia cruzada:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_i -\left(1 - y^{(i)}\right) \ln \left[1 - \sigma\left(f\left[x^{(i)}, \phi\right]\right)\right] - y^{(i)} \ln \left[\sigma\left(f\left[x^{(i)}, \phi\right]\right)\right] \right]$$

- $\sigma(\cdot)$ é a sigmóide que mapeia o logit $f[\cdot, \phi]$ para probabilidade de “real”.
- A primeira parcela penaliza rotular geradas como reais; a segunda penaliza rotular reais como geradas.

Loss do Discriminador (separando reais e geradas)

Para tornar mais evidente que algumas instâncias são geradas ($y^{(j)} = 0$), reescrevemos a loss em duas somas distintas:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_j \left(-\ln \left[1 - \sigma \left(f \left[x^{(j)*}, \phi \right] \right) \right] \right) - \sum_i \left(\ln \left[\sigma \left(f \left[x^{(i)}, \phi \right] \right) \right] \right) \right]$$

- j indexa instâncias **geradas** ($x^{(j)*}$), i indexa instâncias **reais** ($x^{(i)}$).
- O gerador entra apenas no primeiro somatório, por meio das amostras $x^{(j)*} = g[z^{(j)}, \theta]$.

MinMax Loss

Plugando a definição do gerador e **invertendo o objetivo do gerador**, obtemos a formulação adversarial conjunta:

$$\hat{\phi}, \hat{\theta} = \underset{\phi}{\operatorname{argmin}} \underset{\theta}{\operatorname{argmax}} \left[\sum_j \left(-\ln \left[1 - \sigma \left(f \left[g[z^{(j)}, \theta], \phi \right] \right) \right] \right) - \sum_i \left(\ln \left[\sigma \left(f \left[x^{(i)}, \phi \right] \right) \right] \right) \right]$$

MinMax Loss

O discriminador **minimiza** a entropia cruzada (classifica bem) enquanto o gerador **maximiza** o mesmo objetivo (tenta enganar o discriminador).

MinMax Loss – Papéis de gerador e discriminador

$$\hat{\phi}, \hat{\theta} = \underset{\phi}{\operatorname{argmin}} \underset{\theta}{\operatorname{argmax}} \left[\underbrace{\sum_j \left(-\ln \left[1 - \sigma \left(f \left[g[z^{(j)}, \theta], \phi \right] \right) \right] \right)}_{\text{Gerador + Discriminador}} - \underbrace{\sum_i \left(\ln \left[\sigma \left(f \left[x^{(i)}, \phi \right] \right) \right] \right)}_{\text{Discriminador}} \right]$$

- O **discriminador** recebe gradientes dos dois termos: quer classificar bem reais e geradas.
- O **gerador** só interage com o primeiro termo: só se importa em enganar o discriminador sobre as amostras sintéticas.

MinMax Loss – Loss por rede

Na prática, treinamos as redes alternadamente, cada uma otimizando sua própria loss:

Loss do discriminador (minimizar em ϕ)

$$L[\phi] = \sum_j \left(-\ln \left[1 - \sigma \left(f \left[g[z^{(j)}, \theta], \phi \right] \right) \right] \right) - \sum_i \left(\ln \left[\sigma \left(f \left[x^{(i)}, \phi \right] \right) \right] \right)$$

Loss do gerador (minimizar em θ)

$$L[\theta] = \sum_j \left(\ln \left[1 - \sigma \left(f \left[g[z^{(j)}, \theta], \phi \right] \right) \right] \right)$$

Non-saturating Loss – Truque prático do gerador

No início do treino, o discriminador vence facilmente: $\sigma(f[g(z), \phi]) \approx 0$. A loss $\ln[1 - \sigma(\cdot)]$ vira quase plana em 0, e o gradiente que chega ao gerador é minúsculo.

Substituição usada na prática

$$L_{\text{NS}}[\theta] = \sum_j \left(-\ln \left[\sigma \left(f \left[g[z^{(j)}, \theta], \phi \right] \right) \right] \right)$$

- Em vez de minimizar a probabilidade de ser pega, o gerador maximiza a probabilidade de parecer real.
- Mesmo ponto fixo (equilíbrio em $\sigma = \frac{1}{2}$), mas gradiente saudável quando o gerador é fraco.
- Já estava na proposta original de (Goodfellow et al., 2014) e é o default em praticamente todas as implementações.

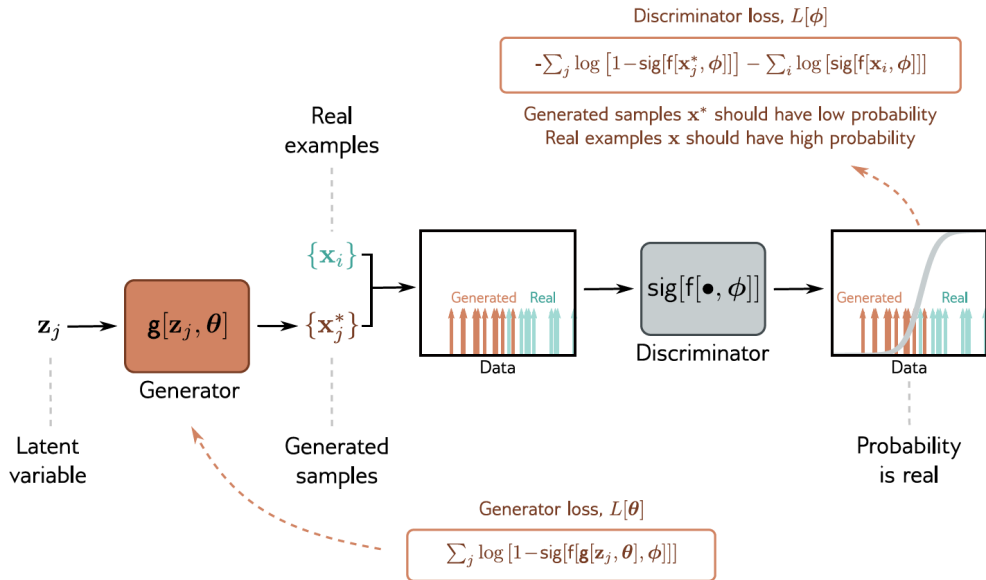
MinMax Loss – Algoritmo de treino

Treinamos as redes **alternadamente**, em passos de gradiente:

Iteração de treino

1. Amostre minibatch de reais $\{x^{(i)}\}_{i=1}^m$ e ruído $\{z^{(j)}\}_{j=1}^m$.
 2. Repita k vezes: atualize o discriminador $\phi \leftarrow \phi - \eta_{\phi} \nabla_{\phi} L[\phi]$.
 3. Amostre novo ruído $\{z^{(j)}\}_{j=1}^m$ e atualize o gerador $\theta \leftarrow \theta - \eta_{\theta} \nabla_{\theta} L[\theta]$.
- No paper original $k = 1$. Em *WGAN* veremos $k = 5$ (queremos crítico próximo do ótimo).
 - Otimizador típico: ADAM com $\beta_1 = 0,5$ ($\beta_1 = 0,9$ desestabiliza GANs).

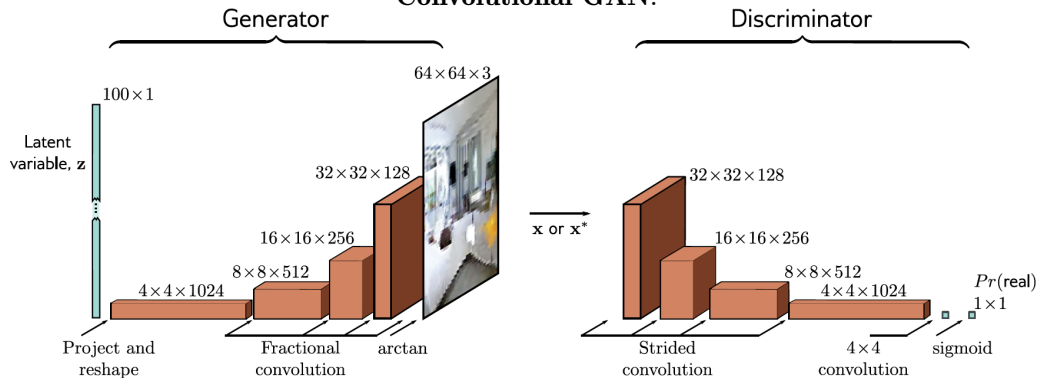
MinMax Loss – Pipeline de treinamento



Overview

1. O que são GANs
2. MinMax Loss
- 3. DCGAN**
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

A primeira arquitetura a popularizar GANs para imagens naturais: **Deep Convolutional GAN**.



- Gerador usa **convoluções fracionárias** (upsampling) partindo de um vetor latente 100×1 .
- Discriminador usa **convoluções com stride** para reduzir a resolução.

DCGAN – Receita arquitetural

O paper destila um conjunto de guidelines que dominaram os anos seguintes:

- **Sem *pooling*:** troque por convoluções com *stride* (D) e convoluções fracionárias (G).
- **Sem camadas *fully connected*** ocultas em ambas as redes.
- **BatchNorm** em D e G, exceto na saída de G e na entrada de D (estabiliza o treino sem distorcer a saída).
- **Ativações:** ReLU em todas as camadas de G (Tanh na saída); LeakyReLU em todas as camadas de D.
- Vetor latente $z \in \mathbb{R}^{100}$, normal padrão.

Por que importa

Antes da DCGAN, treinar GAN convolucional era considerado quase impossível. Esses cinco pontos viraram baseline para praticamente todas as arquiteturas posteriores.

DCGAN – Amostras geradas



Figure 15.4 Synthesized images from the DCGAN model. a) Random samples drawn from DCGAN trained on a faces dataset. b) Random samples using the ImageNet database (see figure 10.15). c) Random samples drawn from the LSUN scene understanding dataset. Adapted from Radford et al. (2015).

Amostras sintéticas geradas por DCGANs treinadas em três *datasets*: (a) faces, (b) ImageNet, (c) LSUN (cenais). Adaptado de (Radford, Metz e Chintala, 2015).

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
- 4. Colapso para a Moda**
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Mode Collapse

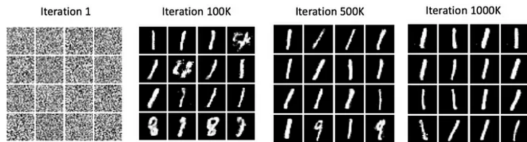
É comum que o gerador “**colapse para a moda**” da distribuição, gerando instâncias sem diversidade.



Figure 15.5 Mode collapse. Synthesized images from a GAN trained on the LSUN scene understanding dataset using an MLP generator with a similar number of parameters and layers to the DCGAN. The samples are low quality, and many are similar. Adapted from Arjovsky et al. (2017).

GAN em LSUN: cenas muito similares.

O motivo disso fica aparente ao analisarmos a função de custo.



MNIST ao longo das iterações: diversidade some.

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
- 5. Análise da Loss**
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Análise da Loss – Normalização

Partimos da loss do discriminador e **dividimos cada somatório** pelo número de instâncias (geradas J ou reais I):

$$L[\phi] = \sum_j \left(\ln \left[1 - \sigma \left(f[x^{(j)*}, \phi] \right) \right] \right) + \sum_i \left(\ln \left[\sigma \left(f[x^{(i)}, \phi] \right) \right] \right)$$

$$L[\phi] \approx \frac{1}{J} \sum_j \left(\ln \left[1 - \sigma \left(f[x^{(j)*}, \phi] \right) \right] \right) + \frac{1}{I} \sum_i \left(\ln \left[\sigma \left(f[x^{(i)}, \phi] \right) \right] \right)$$

Isso não altera o mínimo (multiplicar por constante não muda argmin), mas prepara o próximo passo.

Análise da Loss – Esperança/Expectativa

Ao dividir pelo número de instâncias, cada termo vira uma **esperança** sobre a distribuição correspondente:

$$L[\phi] = \mathbb{E}_{x^*} [\ln[1 - \sigma(f[x^*, \phi])]] + \mathbb{E}_x [\ln[\sigma(f[x, \phi])]]$$

- $\mathbb{E}_{x^*}$: esperança sobre a distribuição gerada $P(x^*)$.
- $\mathbb{E}_x$: esperança sobre a distribuição real $P(x)$.
- Essa reescrita muda o ponto de vista: deixamos de falar de amostras e passamos a falar de distribuições.

Análise da Loss – Integrais

Aplicando a definição de esperança, ficamos com **duas integrais**:

$$L[\phi] = \int P(x^*) \ln[1 - \sigma(f[x^*, \phi])] dx^* + \int P(x) \ln[\sigma(f[x, \phi])] dx$$

- Os integrandos dependem do discriminador $\sigma(f[\cdot, \phi])$.
- Vamos substituir o discriminador pela expressão ótima (que minimiza essa loss).

Análise da Loss – Discriminador ótimo

Assumindo $I = J$, o discriminador ótimo para uma amostra desconhecida $\tilde{x}$ é:

$$P(\text{real} \mid \tilde{x}) = \sigma[f[\tilde{x}, \phi]] = \frac{P(\tilde{x} \mid \text{real})}{P(\tilde{x} \mid \text{real}) + P(\tilde{x} \mid \text{gen})} = \frac{P(x)}{P(x) + P(x^*)}$$

- Obtido por **regra de Bayes** assumindo que $P(\text{real}) = P(\text{gen}) = \frac{1}{2}$.
- É o classificador “perfeito” que só depende das densidades verdadeira e gerada.

Análise da Loss – Substituindo o discriminador ótimo

Substituindo $\sigma(f[x, \phi]) = \frac{P(x)}{P(x) + P(x^*)}$ na loss:

$$L[\phi] = \int P(x^*) \ln \left[\frac{P(x^*)}{P(x) + P(x^*)} \right] dx^* + \int P(x) \ln \left[\frac{P(x)}{P(x) + P(x^*)} \right] dx$$

Os dois termos dentro dos logaritmos viraram **razões de densidades**, que é exatamente a forma de uma divergência KL.

Análise da Loss – Truque do fator 2

Por conveniência matemática, multiplicamos e dividimos por 2 dentro do logaritmo (não altera o argmin):

$$L[\phi] = \frac{1}{2} \int P(x^*) \ln \left[\frac{2P(x^*)}{P(x) + P(x^*)} \right] dx^* + \frac{1}{2} \int P(x) \ln \left[\frac{2P(x)}{P(x) + P(x^*)} \right] dx$$

Agora cada termo é exatamente uma divergência KL entre uma distribuição e a mistura $\frac{P(x^*)+P(x)}{2}$.

Análise da Loss – Reconhecendo KL

Cada parcela tem a forma da **KL Divergence**:

$$D_{KL}[p(x) \parallel q(x)] = \int p(x) \ln \left[\frac{p(x)}{q(x)} \right] dx$$

Portanto:

$$L[\phi] = \frac{1}{2} D_{KL} \left[P(x^*) \parallel \frac{P(x^*) + P(x)}{2} \right] + \frac{1}{2} D_{KL} \left[P(x) \parallel \frac{P(x^*) + P(x)}{2} \right]$$

Análise da Loss – *Jensen–Shannon Divergence*

Obtemos exatamente a definição da **Jensen–Shannon Divergence**:

$$D_{JS}[P(x^*) \parallel P(x)] = \frac{1}{2}D_{KL}\left[P(x^*) \parallel \frac{P(x^*)+P(x)}{2}\right] + \frac{1}{2}D_{KL}\left[P(x) \parallel \frac{P(x^*)+P(x)}{2}\right]$$

Interpretação

Com discriminador ótimo, a loss de uma GAN é, a menos de constante, a **divergência de Jensen–Shannon** entre a distribuição real e a gerada.

Análise da Loss – Qualidade vs. Cobertura

$$D_{JS}[P(x^*)||P(x)] = \underbrace{\frac{1}{2}D_{KL}\left[P(x^*)\left\|\frac{P(x^*)+P(x)}{2}\right.\right]}_{\text{Qualidade}} + \underbrace{\frac{1}{2}D_{KL}\left[P(x)\left\|\frac{P(x^*)+P(x)}{2}\right.\right]}_{\text{Cobertura}}$$

- **Primeiro termo (qualidade)**: penaliza regiões em que $P(x^*)$ tem probabilidade alta mas a mistura não tem — ou seja, amostras geradas implausíveis.
- **Segundo termo (cobertura)**: penaliza regiões em que $P(x)$ tem probabilidade alta mas a mistura não — ou seja, modas ignoradas pelo gerador.

Pergunta

Qual dos dois termos o gerador consegue de fato controlar?

Análise da Loss – Por que há colapso de moda

$$D_{JS}[P(x^*) \| P(x)] = \frac{1}{2} D_{KL}\left[P(x^*) \left\| \frac{P(x^*) + P(x)}{2}\right.\right] + \underbrace{\frac{1}{2} D_{KL}\left[P(x) \left\| \frac{P(x^*) + P(x)}{2}\right.\right]}_{\text{depende fracamente de } P(x^*)}$$

Consequência

Quando $P(x^*) \ll P(x)$ em uma região, o **segundo termo** é dominado por $\frac{P(x)}{P(x)/2}$ e fica insensível a aumentos pequenos em $P(x^*)$.

O gerador é **incentivado muito mais fortemente a qualidade** que a cobertura $\rightarrow$ **colapso para a moda**.

Análise da Loss – Recapitulando

A cadeia de manipulações que aplicamos:

1. Loss original = entropia cruzada binária do discriminador.
2. Normalizamos cada soma $\rightarrow$ esperanças sobre $P(x)$ e $P(x^*)$.
3. Esperanças $\rightarrow$ integrais.
4. Substituímos $\sigma(f)$ pelo discriminador ótimo $\frac{P(x)}{P(x)+P(x^*)}$.
5. Multiplicamos e dividimos por 2 dentro do logaritmo.
6. Cada termo virou uma KL contra a mistura $\frac{P(x)+P(x^*)}{2}$.
7. Somando, obtemos $D_{JS}[P(x^*) \parallel P(x)]$.

Moral

Treinar uma GAN com discriminador ótimo equivale a minimizar Jensen–Shannon. As fragilidades dessa divergência (assimetria qualidade vs cobertura, comportamento em suportes disjuntos) viram fragilidades do treinamento.

Exemplo Numérico

O Que Cada Valor Representa

Antes de calcular o D_{JS} , vamos ancorar cada símbolo em algo **concreto**. Imagine que estamos treinando uma GAN para gerar imagens em três categorias: gato, cão e pássaro. Identifique cada uma com a , b , c .

- $P(x)$: fração das imagens **reais** (no dataset) em cada categoria. É fixo, a verdade.
- $P^*(x)$: fração das imagens **geradas** pelo modelo atual em cada categoria. Muda conforme o gerador treina.
- $D^*(x)$: probabilidade que o **discriminador ótimo** atribui à categoria x de ser real. Onde P domina, D^* tende a 1; onde P^* domina, D^* tende a 0.
- $M(x) = \frac{1}{2}(P + P^*)$: distribuição de **mistura**, o referencial neutro contra o qual ambas as divergências KL são medidas.
- $D_{JS}(P^*||P)$: o **único número** que resume a distância entre o que o gerador faz hoje e o que deveria fazer. Zero $\Rightarrow$ gerador perfeito; $\ln 2 \Rightarrow$ suportes disjuntos.

Exemplo Numérico: D^* e D_{JS} em 1D

Suporte $\{a, b, c\}$. Real $P = (0,5, 0,3, 0,2)$; gerada $P^* = (0,2, 0,5, 0,3)$.

Discriminador ótimo

$$D^*(x) = \frac{P(x)}{P(x) + P^*(x)}:$$

$$D^* \approx (0,71, 0,38, 0,40)$$

Mistura $M = \frac{1}{2}(P + P^*)$:

$$M = (0,35, 0,40, 0,25)$$

$$D_{JS}(P^* \| P) = \frac{1}{2}(0,047 + 0,054) \approx \mathbf{0,05} \text{ nats}$$

x	$P \ln(P/M)$	$P^* \ln(P^*/M)$
a	+0,178	-0,112
b	-0,086	+0,112
c	-0,045	+0,055
Σ	$D_{KL}(P \ M) = 0,047$	$D_{KL}(P^* \ M) = 0,054$

Suporte disjunto

Se $P(c) = 0$, o termo $P \ln(P/M)$ em c vai a 0, mas $P^* \ln(P^*/M)$ pesa fortemente ali $\rightarrow D_{JS} \rightarrow \ln 2 \approx 0,69$ (máximo). É a fronteira que motiva Wasserstein GAN, próxima seção.

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
- 6. Vanishing Gradient**
7. Wasserstein GAN
8. Tricks de Treinamento
9. Panorama Geral

Por que distribuições disjuntas são a regra

- **Hipótese da variedade:** dados naturais (imagens, áudio, texto) vivem em uma variedade de baixa dimensão mergulhada num espaço ambiente de dimensão muito maior.
- O gerador também produz amostras sobre uma variedade: a imagem da função $g[\cdot, \theta]$ aplicada ao latente $z \in \mathbb{R}^d$, com $d \ll D$.
- **Duas variedades de dimensão baixa em $\mathbb{R}^D$ genericamente têm interseção de medida zero.**
- Logo, no início do treino $P(x)$ e $P(x^*)$ tendem a ter suportes quase disjuntos, e a D_{JS} fica congelada em $\ln 2$.

Conclusão

Distribuições disjuntas **não** são caso patológico raro: são a situação típica para dados de alta dimensão. Por isso o problema a seguir aparece em quase todo treino de GAN.

Vanishing Gradient – Motivação

- Vimos que, no caso de discriminador ótimo, a loss minimiza uma **distância** (D_{JS}) entre instâncias reais e geradas.
- Contudo, essa abordagem traz problemas:
 - Se as distribuições são **disjuntas**, a D_{JS} assume valor constante $\ln 2$ e a D_{KL} diverge.
 - Nesse caso, nenhuma “pequena” mudança no gerador melhora a loss — gradientes desaparecem.

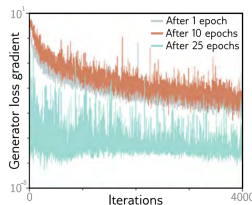
Relembrando KL

$$D_{KL}[p(x)||q(x)] = \int p(x) \ln \left[\frac{p(x)}{q(x)} \right] dx$$

Diverge sempre que existe x com $p(x) > 0$ e $q(x) = 0$.

Vanishing Gradient – Evidência empírica⁶

- O discriminador é congelado após 1, 10 e 25 épocas e o gerador continua treinando.
- Conforme o discriminador fica “bom demais”, o gradiente que chega ao gerador despenca (escala log).
- Sintoma típico: treinamento instável, no qual o discriminador precisa ser deliberadamente enfraquecido para que o gerador aprenda.



Adaptado de (Arjovsky e Bottou, 2017).

Figure 15.7 Vanishing gradients in the generator of a DCGAN. The generator is frozen after 1, 10, and 25 epochs, and the discriminator is trained further. The gradient of the generator decreases rapidly (note log scale); if the discriminator becomes too accurate, the gradients for the generator vanish. Adapted from Arjovsky & Bottou (2017).

⁶Martin Arjovsky e Léon Bottou. “Towards Principled Methods for Training Generative Adversarial Networks”. Em: International Conference on Learning Representations. 2017.

Motivação para mudar a distância

- Vimos que a loss original de GANs pode ser interpretada como uma **distância entre distribuições de probabilidade**.
- Porém, é uma distância com propriedades **inadequadas para otimização**:
 - Distribuições disjuntas têm D_{KL} infinita e D_{JS} constante.
 - Gradientes somem rapidamente.
- Vamos escolher uma distância com propriedades mais adequadas para otimização: a distância de **Wasserstein**.

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
- 7. Wasserstein GAN**
8. Tricks de Treinamento
9. Panorama Geral

Wasserstein Distance⁷

- Também conhecida como *Earth Mover's Distance* (para distribuições discretas).
- Definida pela **quantidade de trabalho** necessária para transportar a massa de probabilidade de uma distribuição até transformá-la em outra.
 - Trabalho = massa $\times$ distância movida.
- Oferece **gradientes informativos** mesmo quando as distribuições são disjuntas.

⁷Martin Arjovsky, Soumith Chintala e Léon Bottou. “Wasserstein Generative Adversarial Networks”. Em: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. 2017, pp. 214–223.

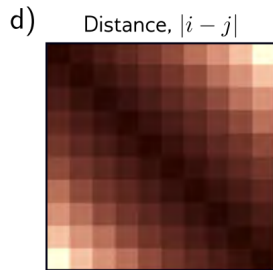
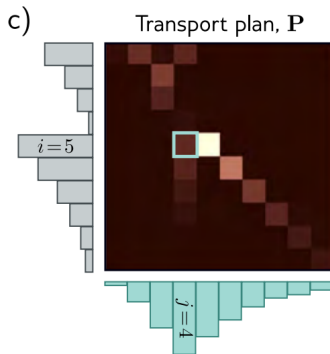
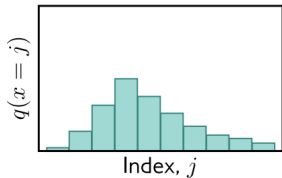
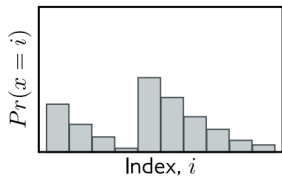
Wasserstein Distance – Caso discreto

Definição

$$D_w[Pr(x) \parallel Qr(x)] = \min_P \left[\sum_{i,j} P_{ij} |i - j| \right]$$

- P é o “**plano de transporte**”: P_{ij} indica quanta massa vai de i (origem) para j (destino).
- Sujeito às restrições:
 - $\sum_j P_{ij} = Pr(x = i)$
 - $\sum_i P_{ij} = Qr(x = j)$
 - $P_{ij} \geq 0$

Wasserstein Distance – Visualização



$$\text{Wasserstein distance} = \sum \mathbf{P} \cdot |i - j|$$

Distribuições Pr , Qr ; plano de transporte $\mathbf{P}$; matriz de custo $|i - j|$. Distância total = $\sum \mathbf{P} \cdot |i - j|$.

Wasserstein Distance – Formulação dual

- Para obter o valor de D_w , precisamos resolver um problema de **programação linear** em P .
- Isso não é prático durante o treinamento de GANs.
- Para esse problema específico, a formulação **dual** é mais útil.

Wasserstein Distance – Forma dual

Reescrevendo em sua forma dual, temos uma nova variável de otimização f que **maximizamos** sob a restrição de que valores adjacentes não se alterem em mais de uma unidade:

$$D_w[Pr(x) \parallel Qr(x)] = \max_f \left[\sum_i Pr(x = i) f_i - \sum_j Qr(x = j) f_j \right]$$

Sob a restrição: $|f_{i+1} - f_i| < 1$ (restrição de **1-Lipschitz** no caso contínuo).

Wasserstein GAN – Loss

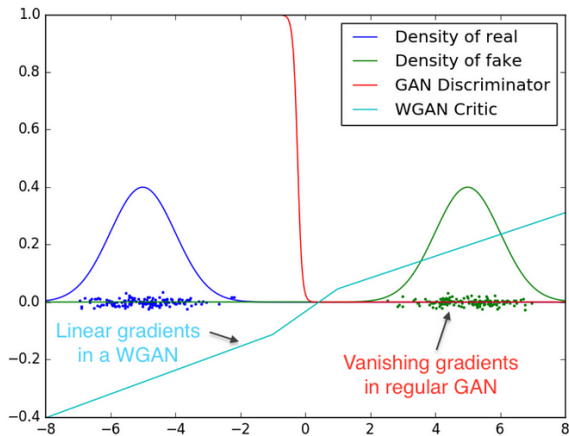
Considerando que as distribuições são aproximadas pelas instâncias reais e geradas, obtemos a loss do **WGAN**:

$$L[\phi, \theta] = \sum_j f[g[z^{(j)}, \theta], \phi] - \sum_i f[x^{(i)}, \phi]$$

Sujeito à restrição: $\left| \frac{\partial f(x, \phi)}{\partial x} \right| < 1$

- Não há mais sigmóide: f é um **crítico**, não um classificador.
- A restrição de Lipschitz é imposta via weight clipping (WGAN) ou gradient penalty[5].

WGAN – Gradientes em distribuições disjuntas



Adaptado de Jonathan Hui (Medium, 2018).

- Discriminador **saturado** $\Rightarrow$ gradientes nulos.
- Crítico WGAN produz gradientes **lineares** mesmo em regiões disjuntas $\Rightarrow$ treinamento muito mais estável.
- A restrição de Lipschitz é a chave: evita que o crítico “exploda” para separar as distribuições.

WGAN – Algoritmo

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

Algoritmo original do WGAN com weight clipping. Adaptado de (Arjovsky, Chintala e Bottou, 2017).

WGAN-GP – Gradient Penalty⁸

Weight clipping impõe Lipschitz, mas reduz a capacidade do crítico (pesos saturam em $\pm c$). **WGAN-GP** substitui o clipping por um termo de penalidade na loss:

$$L_{\text{GP}} = \underbrace{\sum_j f[g(z^{(j)}), \phi] - \sum_i f[x^{(i)}, \phi]}_{\text{loss WGAN original}} + \lambda \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} f[\hat{x}, \phi]\|_2 - 1)^2 \right]$$

- $\hat{x} = \alpha x + (1 - \alpha) x^*$ com $\alpha \sim \mathcal{U}(0, 1)$: pontos interpolados entre real e gerada.
- Penaliza norma do gradiente diferente de 1, isto é, Lipschitz é atingida na intensidade certa, não no extremo.
- Permite voltar a usar ADAM e remover BatchNorm do crítico. $\lambda = 10$ funciona bem na prática.
- Hoje é o default quando se fala em “WGAN”.

⁸Ishaan Gulrajani et al. “Improved Training of Wasserstein GANs”. Em: Advances in Neural Information Processing Systems. Vol. 30. 2017.

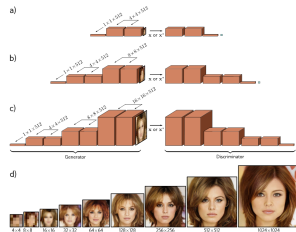
Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
- 8. Tricks de Treinamento**
9. Panorama Geral

Trick 1: Progressive growing⁹

- Começamos treinando GAN em resolução baixa (4×4).
- À medida que o treino avança, vamos **adicionando camadas** para dobrar a resolução até 1024×1024 .
- Transições suaves entre resoluções evitam instabilidade.
- Resultado: imagens de altíssima qualidade (faces CelebA-HQ).

Trick 1: Progressive growing



⁹Tero Karras et al. "Progressive Growing of GANs for Improved Quality, Stability, and Variation". [Em: International Conference on Learning Representations](#). 2018.

Trick 2: Minibatch discrimination¹⁰

- Adicione **estatísticas do *minibatch*** como entrada extra do discriminador.
- Exemplos: variância intra-batch, distâncias par-a-par entre amostras.
- **Efeito:** força o batch gerado a ter variabilidade semelhante ao batch real.
 - Ataca diretamente o colapso de moda: amostras muito parecidas passam a ser detectáveis.

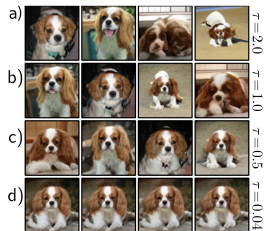
¹⁰Tim Salimans et al. “Improved Techniques for Training GANs”. Em: [Advances in Neural Information Processing Systems](#). Vol. 29. 2016.

Trick 3: Truncation¹¹

- Na hora de amostrar, só aceite z cujo módulo seja menor que um limiar τ .
- τ grande $\rightarrow$ mais diversidade, menos qualidade.
- τ pequeno $\rightarrow$ mais qualidade, menos diversidade.
- Permite controlar o trade-off qualidade/diversidade sem retreinar a rede.

Trick 3: Truncation

Only choose random values of latent variables that are less than a threshold τ .



¹¹Andrew Brock, Jeff Donahue e Karen Simonyan. “Large Scale GAN Training for High Fidelity Natural Image Synthesis”. Em: International Conference on Learning Representations. 2019.

Overview

1. O que são GANs
2. MinMax Loss
3. DCGAN
4. Colapso para a Moda
5. Análise da Loss
6. Vanishing Gradient
7. Wasserstein GAN
8. Tricks de Treinamento
- 9. Panorama Geral**

Como avaliar uma GAN?

Não há log-verossimilhança para reportar. Métricas usuais:

- **Inception Score**¹²

$$\text{IS} = \exp(\mathbb{E}_{x^*} [D_{KL}(p(y | x^*) \| p(y))])$$

Recompensa imagens de classe nítida ($p(y | x^*)$ concentrada) e diversidade entre amostras ($p(y)$ espalhada). Limitação: depende do classificador (Inception em ImageNet).

- **Fréchet Inception Distance (FID)**¹³: ajusta gaussianas em features do Inception e calcula

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{tr}(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g})$$

Mais robusto que IS, é o padrão atual; valores menores são melhores.

Regra de bolso

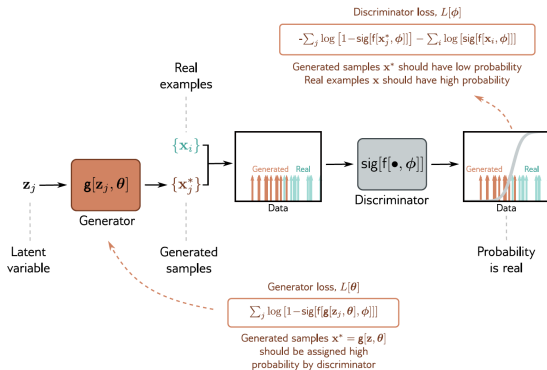
Reporte **FID** e complemente com inspeção visual. Cuidado: ambas as métricas são sensíveis ao tamanho da amostra.

¹²Tim Salimans et al. “Improved Techniques for Training GANs”. Em: [Advances in Neural Information Processing Systems](#). Vol. 29. 2016.

¹³Martin Heusel et al. “GAN Trained by a Two-Stage Wasserstein Process”. In: [International Conference on Learning Representations](#). 2017.

TL;DR sobre GANs

- Existem diversas arquiteturas com algum controle sobre as imagens geradas:
 - Conditional GANs*
 - StyleGAN, BigGAN, ...*
- Pontos fortes**
 - Resultados bastante realistas.
 - Inferência “rápida” (1 passagem pelo gerador).
- Pontos fracos**
 - Difíceis de treinar.
 - Baixa cobertura de possibilidades (mode collapse).



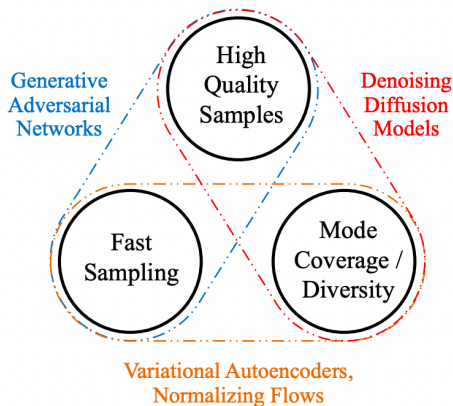
O trilema dos modelos geradores

Todo modelo gerador precisa trocar três objetivos:

- **Qualidade** das amostras.
- **Cobertura** / diversidade de modos.
- **Velocidade** de amostragem.

Posicionamento

- *GANs*: qualidade + velocidade.
- *Diffusion Models*: qualidade + cobertura.
- *VAEs/Normalizing Flows*: velocidade + cobertura.



Leituras Recomendadas

- Capítulo 15: Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Ian Goodfellow et al. “Generative Adversarial Nets”. Em: Advances in Neural Information Processing Systems. Vol. 27. 2014
- Alec Radford, Luke Metz e Soumith Chintala. “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. Em: arXiv preprint arXiv:1511.06434 (2015)
- Martin Arjovsky, Soumith Chintala e Léon Bottou. “Wasserstein Generative Adversarial Networks”. Em: Proceedings of the 34th International Conference on Machine Learning. Vol. 70. 2017, pp. 214–223
- <https://vincentherrmann.github.io/blog/wasserstein/>

Referências I

- [1] Martin Arjovsky e Léon Bottou. “Towards Principled Methods for Training Generative Adversarial Networks”. Em: [International Conference on Learning Representations](#). 2017.
- [2] Martin Arjovsky, Soumith Chintala e Léon Bottou. “Wasserstein Generative Adversarial Networks”. Em: [Proceedings of the 34th International Conference on Machine Learning](#). Vol. 70. 2017, pp. 214–223.
- [3] Andrew Brock, Jeff Donahue e Karen Simonyan. “Large Scale GAN Training for High Fidelity Natural Image Synthesis”. Em: [International Conference on Learning Representations](#). 2019.
- [4] Ian Goodfellow et al. “Generative Adversarial Nets”. Em: [Advances in Neural Information Processing Systems](#). Vol. 27. 2014.
- [5] Ishaan Gulrajani et al. “Improved Training of Wasserstein GANs”. Em: [Advances in Neural Information Processing Systems](#). Vol. 30. 2017.
- [6] Martin Heusel et al. “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium”. Em: [Advances in Neural Information Processing Systems](#). Vol. 30. 2017.
- [7] Phillip Isola et al. “Image-to-Image Translation with Conditional Adversarial Networks”. Em: [IEEE Conference on Computer Vision and Pattern Recognition \(CVPR\)](#). 2017.
- [8] Tero Karras, Samuli Laine e Timo Aila. “A Style-Based Generator Architecture for Generative Adversarial Networks”. Em: [IEEE/CVF Conference on Computer Vision and Pattern Recognition \(CVPR\)](#). 2019.
- [9] Tero Karras et al. “Progressive Growing of GANs for Improved Quality, Stability, and Variation”. Em: [International Conference on Learning Representations](#). 2018.
- [10] Simon J. D. Prince. [Understanding Deep Learning](#). MIT Press, 2023.
- [11] Alec Radford, Luke Metz e Soumith Chintala. “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. Em: [arXiv preprint arXiv:1511.06434](#) (2015).

Referências II

- [12] Tim Salimans et al. “Improved Techniques for Training GANs”. Em: Advances in Neural Information Processing Systems. Vol. 29. 2016.
- [13] Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks”. Em: IEEE International Conference on Computer Vision (ICCV). 2017.